

CSA1717-ARTIFICIAL INTELLIGENCE

PRACTICAL LAB EXPERIMENTS

ROKESH(192572100)

Aim :

To write a Python program to implement the **Tic-Tac-Toe game** using a 3×3 game board.

Objective

- To implement a two-player game using Python.
- To represent the game board using a list.
- To check winning conditions.
- To handle player turns and invalid moves.

Algorithm

1. Start the program.
 2. Create a 3×3 board with positions 1–9.
 3. Assign X to Player 1 and O to Player 2.
 4. Display the board.
 5. Ask the current player to select a position.
 6. Check whether the selected position is empty.
 7. Place the player's symbol in that position.
 8. Check all rows, columns, and diagonals for a winner.
 9. If a player wins, display the winner and stop.
 10. If all positions are filled without a winner, declare a draw.
 11. Otherwise, switch to the other player.
-
1. Repeat until the game ends.

Python Code

```
# Tic-Tac-Toe Game
```

```
board = [" " for i in range(9)]
```

```
# Display the board
```

```
def display_board():
```

```
    print()
```

```
    print(" " + board[0] + " | " + board[1] + " | " + board[2])
```

```
    print("---+---+---")
```

```
    print(" " + board[3] + " | " + board[4] + " | " + board[5])
```

```
    print("---+---+---")
```

```
    print(" " + board[6] + " | " + board[7] + " | " + board[8])
```

```
    print()
```

```
# Check whether a player has won
```

```
def check_winner(player):
```

```
    winning_positions = [
```

```
        (0, 1, 2),
```

```
        (3, 4, 5),
```

```
        (6, 7, 8),
```

```
        (0, 3, 6),
```

```
        (1, 4, 7),
```

```
        (2, 5, 8),
```

```
        (0, 4, 8),
```

```
        (2, 4, 6)
```

```
    ]
```

```
    for a, b, c in winning_positions:
```

```
        if board[a] == player and board[b] == player and board[c] == player:
```

```
            return True
```

```
    return False
```

```
# Check whether board is full

def board_full():
    return " " not in board

# Main game

print("TIC-TAC-TOE")

print("Positions are:")

print("1 | 2 | 3")
print("---+---+---")
print("4 | 5 | 6")
print("---+---+---")
print("7 | 8 | 9")

current_player = "X"

while True:
    display_board()

    print("Player", current_player, "turn")

    try:
        position = int(input("Enter position (1-9): "))

        if position < 1 or position > 9:
            print("Invalid position! Enter 1-9.")
            continue

        index = position - 1

        if board[index] != " ":
            print("Position already occupied!")
            continue

        board[index] = current_player

    except ValueError:
        print("Please enter a number.")
        continue
```

```
# Check winner

if check_winner(current_player):
    display_board()
    print("Player", current_player, "wins!")
    break

# Check draw

if board_full():
    display_board()
    print("Game Draw!")
    break

# Change player

if current_player == "X":
    current_player = "O"
else:
    current_player = "X"
```

Sample Input / Output

Suppose Player X enters:

1

Player O:

2

Player X:

4

Player O:

5

Player X:

7

Output

X | O |

---+---+---

X | O |

---+---+---

X | |

Player X wins!

The winning combination is:

1

4

7

which is the **first column**.

Example Game

TIC-TAC-TOE

Positions are:

1 | 2 | 3

---+---+---

4 | 5 | 6

---+---+---

7 | 8 | 9

Player X turn

Enter position (1-9): 1

Player O turn

Enter position (1-9): 5

Player X turn

Enter position (1-9): 2

Player O turn

Enter position (1-9): 8

Player X turn

Enter position (1-9): 3

Player X wins!

Result

Thus, the **Tic-Tac-Toe game was successfully implemented in Python** using a 3×3 board, player turns, winning-condition checking, invalid-input handling, and draw detection.